



DESIGN DOCUMENT

Secure Satellite TV Communications System

Members: William Coleman, Rylan Dansereau, Francesca Saurez-Leon, Guillermo Euspierre
Pena, Jaheim Attri, Aaron Jerez, Aidan Delellis, Francesco Combatti

Advisors: Alexander Perez-Pons, Adonis Sardiñas

Table of Contents

Objectives

1.1 Problem Statement

1.2 Technical Solution

Hardware Design

2.1 Board Overview

2.2 Role in Secure Satellite TV System

Operating Satellite TV System

3.1 System Development

3.2 Encoder Functional Requirements

3.3 Decoder Functional Requirements

Secure Design Principles

4.1 Channel Confidentiality

4.2 Frame Authentication

4.3 Archive Security

Security Measures

5.1 Encrypting Frame Data

5.2 Securing Subscriptions

5.3 Disabling Replay Attacks

5.4 Disabling Power Analysis

5.5 Obfuscating Secret Keys

Objectives

1.1 Problem Statement

Satellite TV broadcasts are typically encrypted to protect content from unauthorized access. However, there are growing concerns about the security of these broadcasts and the risk that their signals could be intercepted or compromised.

If this were to happen, sensitive information could fall into the wrong hands, potentially leading to disastrous consequences. The increasing risk of these breaches highlights the urgent need for a solution:

“How can we securely transmit encrypted satellite TV signals while eliminating all potential vulnerabilities to breaches?”

1.2 Technical Solution

Based on the security risks of existing encrypted broadcasts, we will develop a secure satellite TV communications system. This solution will be designed with core security concepts ensuring the confidentiality and integrity of transmitted broadcasts.

This satellite TV system will be developed with key functionality benchmarks and requirements in tandem with strict security policies. The system's goal is to ensure reliable and secure communication, while balancing the cost of transmission stability and speed.

We can efficiently transmit encrypted satellite TV signals while eliminating all possible vulnerabilities to security breaches by developing a secure encoder software for outgoing transmissions alongside an embedded system to securely decode broadcast frames.

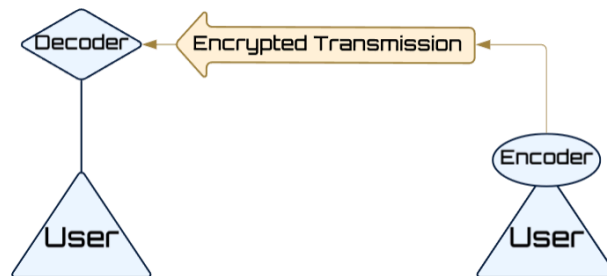


Figure 1.2.a

Secure Satellite TV Solution

Hardware Design

2.1 Board Overview

The **MAX78000FTHR** is an ultra-low-power microcontroller development board designed for AI and embedded security applications. It features the **MAX78000**

microcontroller, integrating an **Arm Cortex-M4** with an FPU (floating-point unit), a **Cortex-M4 DSP** core, and a **hardware AI accelerator**. This combination makes it well-suited for real-time encoding and decoding tasks, critical for secure satellite TV streaming.

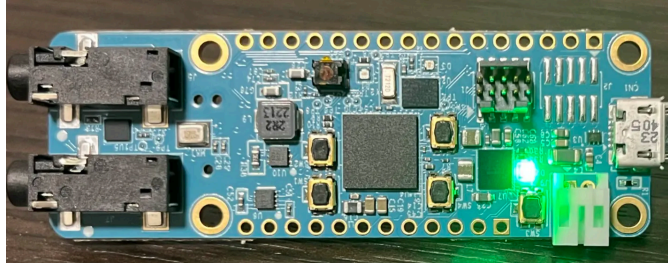


Figure:2.1.a
MAX78000FTHR

Key Hardware Features

- Processor:
 - ◆ Arm Cortex-M4 with FPU
 - ◆ AI Accelerator
- Memory:
 - ◆ 512KB SRAM
 - ◆ 2MB Flash
- Peripherals:
 - ◆ SPI, I²C, UART for communication
 - ◆ GPIOs for hardware interfacing
 - ◆ I²S for audio streaming
- Security Features:
 - ◆ Hardware AES encryption
 - ◆ True Random Number Generator (TRNG)
 - ◆ Secure Boot capabilities

2.2 Role in the Secure Satellite TV System

The **MAX78000FTHR** plays a central role in both encoding and decoding video data, ensuring secure transmission while preventing unauthorized access:

Decoding System (Hardware Decoder on Receiver Side)

- Receives and decrypts satellite TV frames
- Interfaces with the Host Tool to process commands:
- List Channels – Displays available encrypted channels
- Update Channels – Updates security keys and access controls
- Decode Frames – Securely decrypts and displays authorized content

The **MAX78000FTHR**'s built-in security features and low-power AI processing capabilities provide a robust foundation for implementing secure encoding and decoding techniques. By leveraging its cryptographic accelerators and embedded AI capabilities, our design aims to ensure data integrity, confidentiality, and authentication throughout the system.

Operating Satellite TV System

3.1 System Development

There are several steps required to set up and effectively use our satellite TV system: building a virtual environment, setting up/building your own deployment, and finally building our decoder's image and flashing it to the hardware described previously, the **MAX7800FTHR**.

3.1.a Build Environment

The **Build Environment** is a virtual environment (venv), where all dependencies for our satellite TV system are installed. The installation includes compilers, packages, and all other build tools needed.

At this stage, our design is set up, which includes the encoder, subscription management, secrets generation, and host tools. These host tools establish a basic input/output format for communication between devices and the host computer.

3.1.b Building Deployment

Next, we create a Deployment, which represents a complete set of components generated by our satellite TV provider. To begin this process, you must run the "gen_secrets.py" command to generate essential data for the system.

This data, known as the **Global Secrets**, is used throughout the system and remains unchanged once created. The **Global Secrets** are passed through the **Decoder** to update subscriptions and are also used by the **Encoder**. These secrets ensure that the system remains secure. The **Decoder** is then built by running the "docker build" command, which generates an "image" of the **Decoder** based on the Dockerfile. This image is then transferred to our hardware, the MAX7800FTHR, in the next step.

Decoders and **Encoders** that use the same **Global Secrets** are known as a "Deployment," hence why this is the build Deployment step. **Decoders** and **Encoders** within a validly-built Deployment will be able to work together as a secure, and functional satellite TV system.

3.1.c Flash Decoder image

Once the environment and deployment have been built, you may now “flash the **Decoder**,” which is done by using the relevant **Host Tool**: “ectf.utils.flash.” **Decoders** and **Encoders** are able to be built in any order and may read from the **Global Secrets** as needed.

3.2 Encoder Functional Requirements

Once the system is built, you may start the TV satellite system. In the full deployment, this will include the **Uplink**, **Encoder**, and **Satellite**.

The Encoder is part of the pip-installable Python package “ectf25_design,” which should have been installed while building the virtual environment. It is implemented as a function that accepts the raw frame to encode, the channel it will be sent on, and the timestamp of the frame.

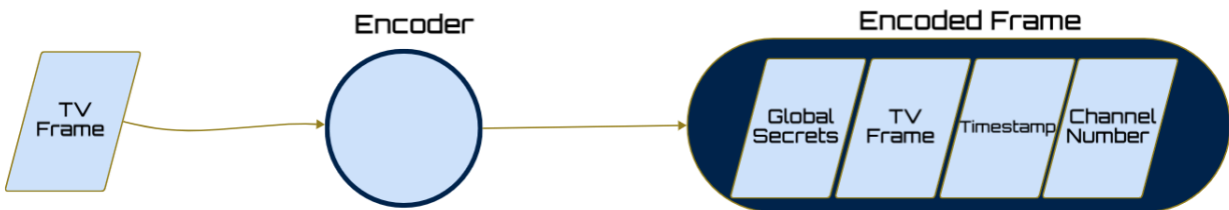


Figure 3.2.a Simple visualization of encoder functionality

3.3 Decoder Functional Requirements

To ensure smooth playback of received frames, the decoder is designed to operate at 10 frames per second, with each frame taking a maximum of 150 milliseconds to process. All communication between the host computer and the decoder is managed through UART, using the `host_messaging.c` library. All of the functionality described below is implemented in the decoder's source code, located in "decoder/src/decoder.c":

- **List Channels**

`list_channels()` returns an array of all channels with an active subscription, along with the respective “start” and “end” timestamps of that subscription’s lifetime. This

array is packaged together with the number of channels it holds before being sent to the host computer.

- ***Update Channel Subscriptions***

`update_subscriptions()` updates the list of channels the decoder currently has an active subscription for. If the host has an expired subscription, that subscription is reactivated and its timestamps modified to reflect the update. Otherwise, a new subscription (along with its timestamp information) is added to the list and activated.

- ***Decode frames***

`decode()` receives a frame packet and prepares it to be sent to the host computer. First, the decoder verifies that the user has a valid subscription for the channel it is receiving data from. Frames are sent using a port from the uplink to the satellite, and from the satellite to the decoder. Afterwards, it implements the “`simple_crypto.c`” library to symmetrically decrypt the frame data and send it to the host computer (packet header not included).

Secure Design Principles

The functionality of the encoder and decoder is irrelevant if the system can easily be compromised. To prevent this, the satellite TV system will be developed with additional core security principles in mind. These take the form of standardized security requirements during the development process to defend against initial attack vectors.

4.1 Channel Confidentiality

The first possible vulnerability to the secure transmission is the unauthorized use of system channel subscriptions.

Figure 4.1.a Attack Vector I

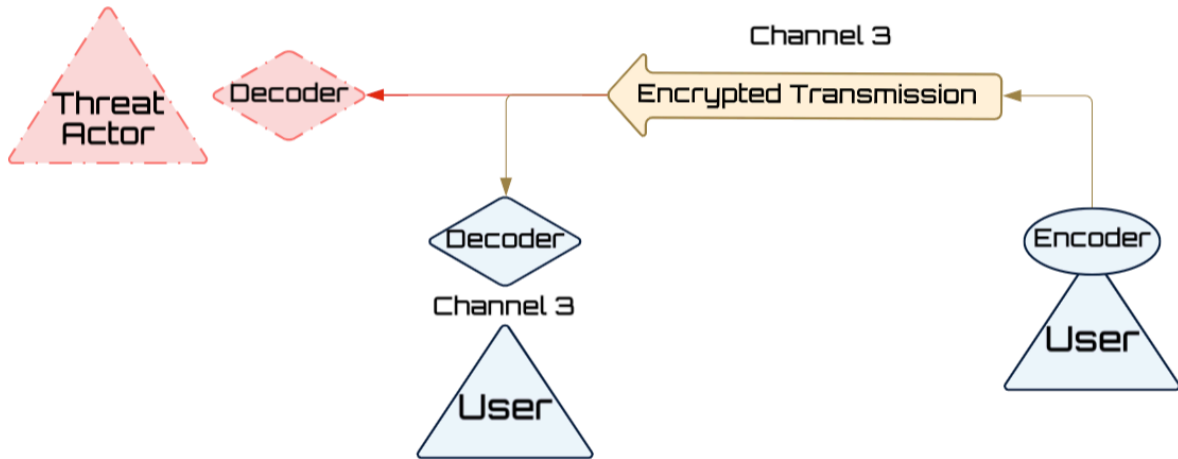


Figure 4.1.b Attack Vector II

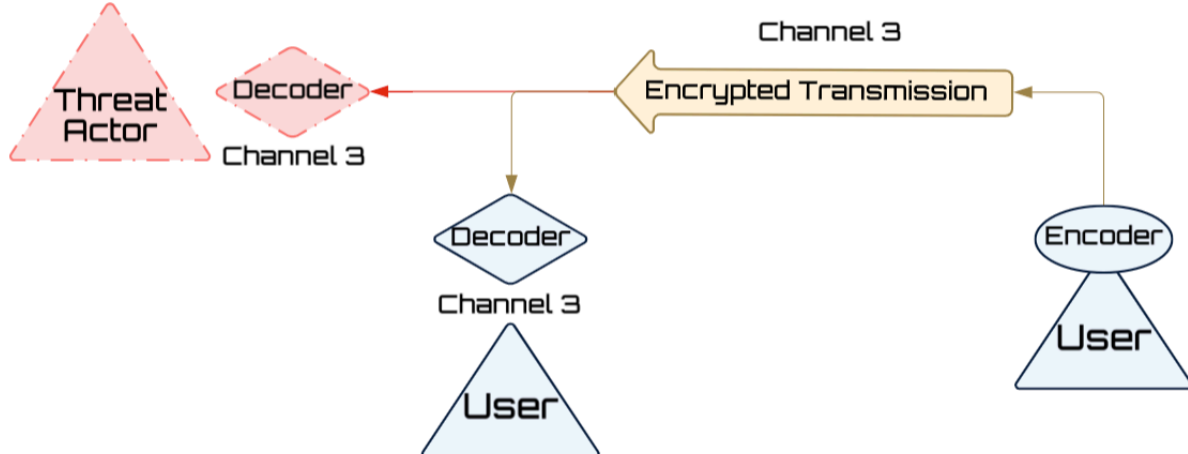
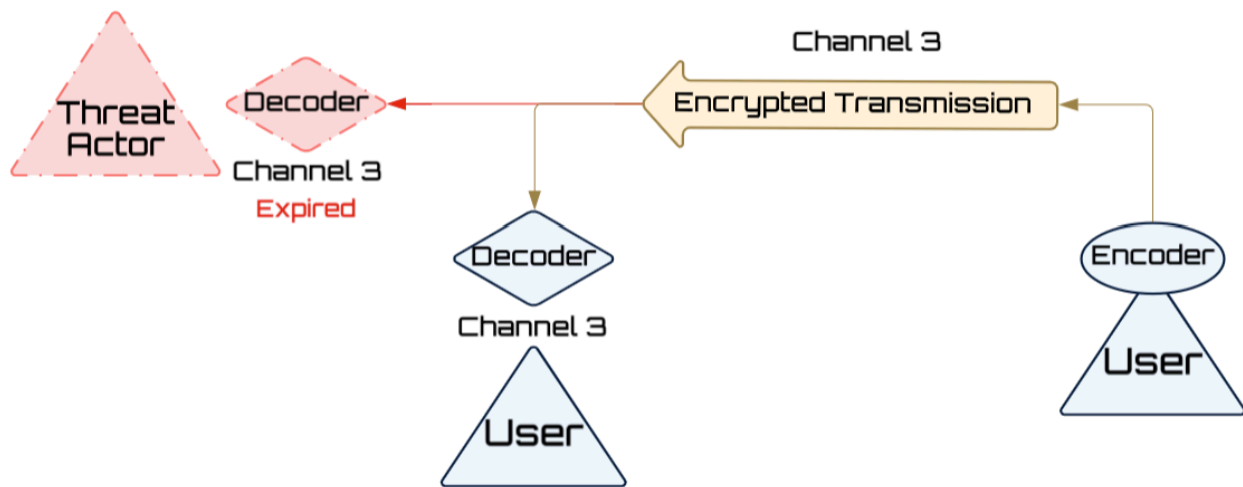


Figure 4.1.c Attack Vector III



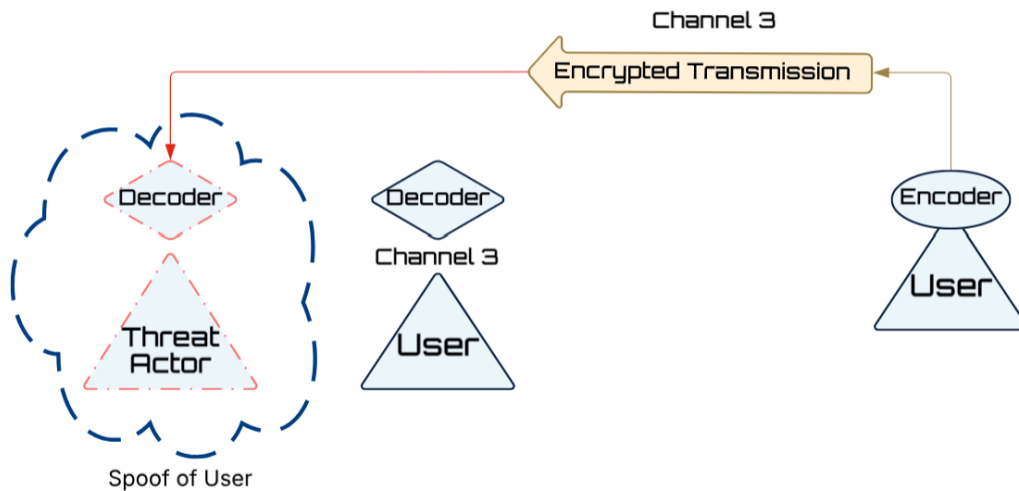
If a threat actor possesses a decoder, it is possible for them to decode any transmission. To prevent this, the first security requirement will be to ensure proper user authorization.

An attacker should not be able to decode TV frames without a Decoder that has a VALID, active subscription to that channel.

4.2 Frame Authentication

In addition to channel subscription, satellite TV communications present additional vulnerabilities. Threat Actors could eliminate the need for channels all together by spoofing the intended recipient's signal.

Figure 4.2.a Attack Vector V



Due to this creative attack, the third security requirement for the development of the TV system will require that the decoder exclusively decode frames from intended satellite systems.

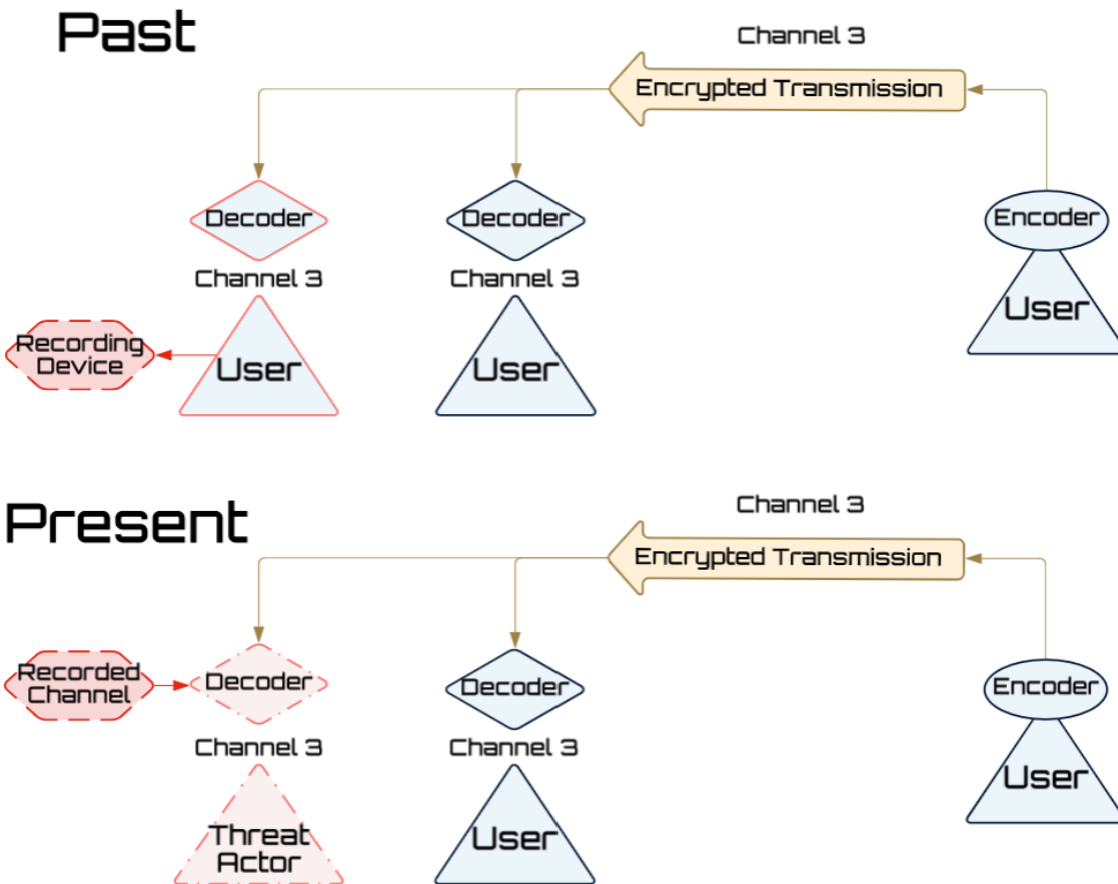
The Decoder should only decode VALID TV FRAMES generated by the satellite system the Decoder was provisioned for.

4.3 Archive Security

In the evolving threat landscape, sometimes threat actors can be currently authorized users. This can be a dangerous situation as authorized users have elevated privileges in comparison to arbitrary attackers.

This situation presents a key vulnerability: The archiving of secure transmissions to be shared with later authorized users, allowing current users to receive past transmissions they did not have authorization for at the time.

Figure 4.3.a Attack Vector IV



To prevent currently authorized users from decoding transmissions they were not authorized to receive, the second security requirement for the satellite TV system will be to issue and enforce timestamps on all transmissions.

The Decoder should only decode frames with strictly monotonically increasing timestamps.

Security Measures

This section outlines the cryptographic mechanisms used to protect sensitive data within the system, including frame data, subscriptions, timestamps, and frame payloads.

5.1 Encrypting Frame Data

To ensure that critical information is not leaked or tampered with during transmission, we encrypt all frame data using AES in CBC mode and verify integrity with HMAC.

5.1.a Symmetric Encryption (AES256-CBC)

Frame data is encrypted using AES (Advanced Encryption Standard) in CBC (Cipher Block Chaining) mode. This mode provides confidentiality by XOR-ing each plaintext block with the previous ciphertext block before encryption.

And, Each frame uses a unique, randomly-generated 16-byte initialization vector (IV) to ensure identical plaintexts produce different ciphertexts. It is stored alongside the ciphertext in the transmitted frame.

The decryption process is implemented in the `aes_decrypt` functions on the decoder and the encryption process is implemented through an AES class imported from PyCryptodome.

The encryption process ensures that even if attackers intercept frames, they cannot recover plaintext content without the encryption key, which is securely stored and never transmitted.

Secured Element: Frame Content

5.1.b Hash-based Message Authentication Code (HMAC)

$$\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m))$$

Where:

- **K:** is the shared secret key
- **m:** is the message (channel, timestamp, IV, encrypted frame)
- **H:** is a cryptographic hash function like SHA-256
- **ipad and opad:** are inner and outer padding constants

Purpose:

The HMAC ensures message integrity and authenticity. When a decoder receives a frame, it computes the HMAC and compares it to the one attached to the frame. If they differ, the frame is rejected and the normal operation ends/exits.

5.2 Securing Subscriptions

Subscriptions grant decoders access to specific data streams. Subscriptions are cryptographically tied to the decoder's identity to prevent spoofing or unauthorized use.

5.2.a HMAC for Subscription Verification

An HMAC (defined in prior section) is computed over the subscription's data (i.e. device ID, start timestamp, end timestamp, channel) using a device-specific key. This key is derived for each decoder by combining the master subscription key with the device ID and hashing the result.

When a subscription update is received, the decoder: (1) verifies that the subscription is intended for this specific device by checking the device ID, (2) recreates the device-specific key using its stored master key and device ID, (3) recomputes the HMAC over the subscription data and compares it to the received signature, and finally (4) only accepts the subscription if signature match exactly.

Anti Tampering Measures

- Constant-time comparison is used to prevent timing attacks when validating signatures.
- A 5-second delay is enforced on invalid signature detection to thwart brute force attempts.
- Subscriptions are stored in flash memory to persist across power cycles.

Purpose:

Ensures that attackers cannot forge valid subscriptions or modify subscription parameters without knowledge of the master subscription key.

5.3 Disabling Replay Attacks

Replay attacks involve capturing and retransmitting previously valid data. Our system prevents this by implementing a timestamp validation strategy and tracking the most recent timestamp (per frame).

5.3.a Subscription Window Validation

Each subscription defines a validity window, with a start and end timestamp. Frames are only accepted if their timestamps fall within the validity window of the corresponding subscription.

This prevents unauthorized access to content from periods before a subscription was active.

5.3.b Recording previous Timestamp

Each frame includes a timestamp that must be monotonically increasing. Our decoder tracks and records the most recent valid timestamp. A new frame must have a strictly greater timestamp to be accepted. Enforcing chronological consistency prevents repeated use of previous valid frames.

The `timestamp_valid()` function performs both monotonically checking and subscription window validation. Another thing to note is that emergency broadcast frames (channel 0) bypass timestamp validation for availability reasons. It is the one channel that all decoders should be able to decode on.

This approach prevents attackers from recording frames during the valid subscription period and replaying them later, or from replaying old frames to try to manipulate the system state.

5.4 Disabling Power Analysis

Power analysis attacks attempt to extract secret keys by measuring a device's power consumption during cryptographic operations. Our implementation includes countermeasures against these "side-channel attacks."

5.4.a Power Consumption Masking

The `add_power_noise()` function introduces random power fluctuations during sensitive operations.

This function: (1) uses the hardware's TRNG (True Random Number Generator) to create unpredictable values, (2) performs data-independent arithmetic operations on these values, (3) introduces timing jitter with variable iteration counts, and finally (4) it includes compiler barriers to prevent optimization that might remove its operations.

5.4.b Strategic Noise Injection

Noise is added before and after key loading operations, additional noise is introduced before and after HMAC computations, and the decryption process includes power masking at critical junctures.

5.4.c Memory Barriers

Memory barriers are used to prevent compiler optimizations that might remove security-critical operations, which ensures that power consumption masking operations are preserved in the final binary.

These techniques make it significantly more difficult for attackers to correlate power consumption patterns with cryptographic operations being performed, protecting secret keys from differential power analysis attacks.

5.5 Obfuscating Secret Keys

Key management is critical to system security. Our implementation uses several techniques to protect cryptographic keys in memory:

5.5.a Key Fragmentation & XOR Obfuscation

Each key is divided into 8-byte fragments, each fragment is XOR'd with a unique mask derived from a build-specific seed. The original key is only reconstructed in memory when needed for cryptographic operations.

5.5.b Secure Memory Cleaning

The `secure_zero_memory()` function implements a three-pass approach to clearing sensitive data:

1. First pass: Overwrite with 0xFF (all ones)
2. Second pass: Overwrite with 0xAA (alternating bits)
3. Third pass: Overwrite with 0x00 (all zeros)

Memory barriers ensure these operations aren't optimized away by the compiler.

5.5.c Key Generation and Storage

Keys are initially generated using a cryptographically secure random number generator. The encoder and decoder never directly expose keys in logs or error messages. Device-specific keys are derived from master keys through secure one-way functions.

5.5.d Key Loading Protection

Key loading functions include randomized timing variations to resist timing attacks, and functions that handle keys are designed to minimize the time keys spend unobfuscated in memory.

These many measures collectively protect against memory dump attacks, cold boot attacks, and other techniques that might attempt to extract cryptographic keys from device memory.

By implementing these security measures, our Satellite TV System provides robust protection against the attack vectors identified in our secure design principles, ensuring confidentiality of frame data, integrity of subscriptions, and protection against replay attacks.